

PUSHING THE EFFICIENCY FRONTIER OF LoRA: A SYSTEMATIC STUDY AND SVD-GUIDED ADAPTIVE RANK COMPRESSION

Qinsi Wang, Yuhan Cheng, Zanwen Fu, Hancheng Ye

Department of Electrical and Computer Engineering
Duke University

ABSTRACT

The rapid scaling of large language models (LLMs) has made full-parameter fine-tuning increasingly impractical in terms of memory, compute, and deployment cost. Parameter-efficient fine-tuning (PEFT) methods address this challenge by updating only a small set of task-specific parameters while keeping the pre-trained backbone frozen. In this report we focus on the widely adopted Low-Rank Adaptation (LoRA) family and conduct a systematic study of three representative variants: standard LoRA, Sparse LoRA, and HiRA. We analyze their modeling assumptions and optimization characteristics, and we compare them empirically on sentiment classification and language modeling benchmarks using a shared DistilBERT backbone under carefully controlled training protocols. Building on these insights, we further propose a simple yet effective SVD-guided adaptive rank compression scheme for LoRA. The method first trains a conventional high-rank LoRA model, then applies singular value decomposition to each adapter to retain only the leading directions that explain at least 90% of the spectral energy, and finally continues fine-tuning in this reduced-rank subspace. Experiments show that our compressed adapters use roughly 30% of the original LoRA parameters while matching or surpassing the baseline performance, thereby pushing the efficiency frontier of LoRA-style PEFT without sacrificing accuracy.

1 INTRODUCTION

The success of transformer-based large language models (LLMs) has been driven by massive model scales and pre-training on web-scale corpora. However, adapting such models to downstream tasks via full-parameter fine-tuning quickly becomes prohibitive: all parameters must be stored, updated, and served, leading to substantial memory, compute, and deployment costs. This limitation has motivated the development of parameter-efficient fine-tuning (PEFT) techniques, which freeze the pre-trained backbone and introduce a lightweight set of trainable parameters for each task. Among these, Low-Rank Adaptation (LoRA) and its variants have emerged as particularly attractive due to their simplicity, strong empirical performance, and ease of integration with existing architectures.

LoRA replaces dense task-specific weight updates with low-rank matrices injected into existing linear layers. By constraining the update to a low-dimensional subspace, LoRA dramatically reduces the number of trainable parameters and the associated optimizer state, while typically achieving accuracy comparable to full fine-tuning. Nonetheless, the global rank hyperparameter remains a key design choice: small ranks can limit expressivity, whereas large ranks diminish efficiency. To address this issue, a number of extensions have been proposed. Sparse LoRA augments the low-rank parameterization with sparsity in the rank dimension, learning an effective rank per layer via gated components and sparsity-inducing regularization. HiRA, in contrast, seeks to overcome the rank bottleneck altogether by modulating the pre-trained weights through a Hadamard (element-wise) product with a low-rank matrix, thereby inducing high-rank updates under a LoRA-like parameter budget.

This report has two main goals. First, we provide a coherent view of these three representative LoRA variants—LoRA, Sparse LoRA, and HiRA—by reviewing their theoretical underpinnings and empirically comparing their trade-offs in terms of accuracy, parameter count, and training cost. To this end, we adopt a common experimental protocol: all methods fine-tune a shared `distilbert-base-uncased` backbone on sentiment classification (SST-2, IMDB) and language modeling (WikiText-2), using identical data preprocessing, optimization hyperparameters, and evaluation metrics. This controlled setup allows us to attribute performance differences primarily to the adaptation mechanism rather than to extraneous implementation details.

Second, inspired by the spectral structure observed in trained LoRA updates, we propose a novel SVD-guided adaptive rank compression scheme. The key idea is to decouple learning from compressing: we first train a conventional high-rank LoRA model, then apply singular value decomposition (SVD) to each LoRA update matrix and retain only the leading singular directions that explain a fixed fraction (e.g., 90%) of the Frobenius-norm energy. The compressed update is reparameterized as a lower-rank LoRA adapter, and the model is fine-tuned for a few additional epochs in this reduced subspace. In our experiments, this procedure reduces the average LoRA rank from 8 to around 2.4 across attention modules—corresponding to roughly 30% of the original adapter parameters—while preserving or improving performance on both SST-2 and IMDB.

In summary, our contributions are threefold:

- We present a unified analysis of LoRA, Sparse LoRA, and HiRA, highlighting their modeling assumptions and expected trade-offs between expressivity and efficiency.
- We perform a fair empirical comparison of these methods on multiple benchmarks with a shared DistilBERT backbone and standardized training hyperparameters.
- We introduce an SVD-guided adaptive rank compression method for LoRA that achieves substantial parameter reduction with negligible or even positive impact on downstream accuracy.

Together, these results shed light on how low-rank and high-rank adapter designs behave in practice and demonstrate that simple, post-hoc spectral compression can significantly enhance the efficiency of LoRA-based PEFT methods.

2 LITERATURE REVIEW

The rapid scaling of transformer-based language models has made full-parameter fine-tuning increasingly impractical. Parameter-efficient fine-tuning (PEFT) methods address this by freezing most pre-trained weights and updating only a small set of task-specific parameters. Among them, Low-Rank Adaptation (LoRA) and two recent extensions—Sparse LoRA (SoRA) and HiRA—have emerged as representative approaches offering different balances of efficiency and expressivity.

2.1 LoRA: LOW-RANK ADAPTATION

LoRA (Hu et al., 2022) constrains the update to a low-rank form,

$$\Delta W = \frac{\alpha}{r} BA,$$

with $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$. This reduces trainable parameters to $r(d_{\text{in}} + d_{\text{out}})$, often less than 1% of full fine-tuning, while preserving the ability to merge the update into W_0 for inference. Despite its simplicity, LoRA often matches full fine-tuning across diverse NLP benchmarks, reflecting the low intrinsic dimensionality of many downstream adaptations.

However, the update rank is capped at r , which restricts expressivity for complex tasks, and LoRA uses the same rank across layers, limiting its ability to allocate capacity where it is most useful.

2.2 SPARSE LoRA (SoRA): ADAPTIVE RANK ALLOCATION

Sparse LoRA (Khaki et al., 2025) mitigates LoRA’s fixed-rank constraint by learning sparsity over an initially larger rank R . It parameterizes the update as

$$\Delta W = B \text{diag}(\mathbf{g}) A,$$

where a gating vector $\mathbf{g}$ selects which rank components are active. An ℓ_0 -style sparsity penalty prunes redundant components during training, yielding an effective rank $r_{\text{eff}} = \|\mathbf{g}\|_0$ after a simple post-training reduction. This adaptive mechanism often improves accuracy under the same parameter budget and reveals where additional capacity is needed.

The flexibility comes with some overhead: SoRA uses proximal gradient updates, requires tuning sparsity schedules, and—despite adapting rank—remains limited to low-rank additive updates.

2.3 HiRA: HADAMARD HIGH-RANK ADAPTATION

HiRA (Huang et al., 2025) addresses the expressivity bottleneck more directly. Instead of adding a low-rank update, it modulates the frozen weights multiplicatively:

$$\Delta W = W_0 \odot (L_1 L_2),$$

where $L_1 L_2$ is low-rank but the Hadamard interaction with W_0 induces a much higher effective rank. This allows HiRA to approach full-rank behavior while using the same number of trainable parameters as LoRA. Empirically, HiRA outperforms LoRA and other PEFT baselines on reasoning-heavy benchmarks.

Its expressivity, however, depends on the structure of W_0 , and the multiplicative parameterization can be more sensitive to optimization dynamics. These limitations motivate variants such as BHRA, which applies Hadamard modulation in a blockwise fashion.

2.4 OTHER LORA EXTENSIONS

Beyond SoRA and HiRA, the LoRA family continues to expand in complementary directions (Zhu et al., 2025; Wang et al., 2025; Chen et al., 2024). QLoRA (Dettmers et al., 2023) combines LoRA with 4-bit quantization to fine-tune models with tens of billions of parameters on a single GPU. Methods such as SP-LoRA, LoSA, and LoRS integrate LoRA with structured sparsity to support fine-tuning on pruned or compressed backbones. High-rank extensions inspired by HiRA, including BHRA, ABBA, and HD-PiSSA, explore more expressive modulation mechanisms. Meanwhile, federated variants such as FedLoRA leverage LoRA adapters as communication-efficient carriers of client-specific updates.

2.5 SUMMARY

LoRA, SoRA, and HiRA represent three distinct points in the PEFT landscape: a simple and widely adopted low-rank baseline, an adaptive extension that learns per-layer capacity, and a high-rank Hadamard modulation method that expands expressivity without increasing parameter count. In our experiments, we compare these approaches under matched budgets to assess their trade-offs in accuracy, efficiency, and cost.

3 IMPLEMENTATION

3.1 EXPERIMENTAL SETUP

In this section we describe the experimental protocol used to compare LoRA, Sparse LoRA, and HiRA in a fair and controlled manner. Detailed configuration of experiments is provided in Appendix A.

3.2 EXPERIMENT RESULTS

Table 1 summarizes the accuracy and macro-averaged F1 scores of LoRA, Sparse LoRA, and HiRA on SST-2 and IMDB. All methods fine-tune a shared DistilBERT backbone under identical training budgets and preprocessing pipelines, enabling a clean comparison of adaptation mechanisms.

Accuracy Results. LoRA remains the strongest overall. Across both datasets, LoRA achieves the highest accuracy and F1. The advantage is modest on SST-2, while IMDB shows a clearer margin, suggesting LoRA better captures longer and more variable sequences.

Table 1: Accuracy and F1 performance of LoRA, Sparse LoRA, and HiRA on SST-2 and IMDB.

Method	SST-2		IMDB	
	ACC	F1	ACC	F1
LoRA	0.9037	0.9058	0.8679	0.8667
Sparse LoRA	0.8899	0.8909	0.8047	0.7879
HiRA	0.8704	0.8723	0.8375	0.8400

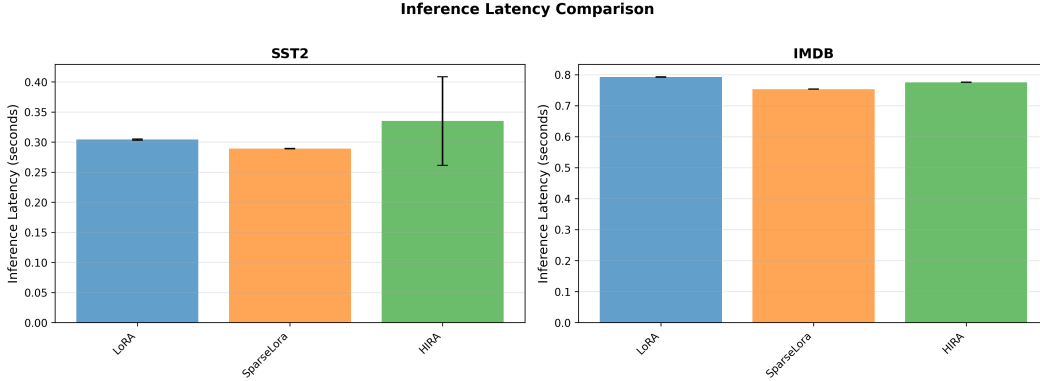


Figure 1: Inference latency of LoRA, Sparse LoRA, and HiRA on SST-2 and IMDB. Error bars denote standard deviation across multiple runs.

Sparse LoRA preserves accuracy on SST-2 but drops notably on IMDB. The sparsity constraint reduces parameter count without hurting performance on short inputs; however, IMDB exhibits an 8–9 point F1 degradation, indicating that aggressive pruning limits representational capacity under longer contexts.

HiRA sits between Sparse LoRA and LoRA, benefiting from high-rank modulation. HiRA surpasses Sparse LoRA on IMDB and approaches LoRA in F1, but does not reach LoRA’s stability within a fixed 3-epoch training schedule.

Inference Latency. Figure 1 presents the average inference time for each method on SST-2 and IMDB. All three methods are measured using the same batch size, tokenizer, and GPU environment.

Latency is broadly similar across LoRA and Sparse LoRA. Although Sparse LoRA contains many pruned parameters, the current PyTorch/HuggingFace execution path does not exploit sparsity for inference speed. As expected, Sparse LoRA achieves latency nearly identical to LoRA on both datasets.

HiRA introduces consistent overhead. HiRA adds multiplicative Hadamard modulation during forward passes, leading to slightly higher latency on both datasets. The overhead is more visible on SST-2, where sequence lengths are short and adapter cost forms a larger fraction of total computation.

IMDB exhibits higher absolute latency for all methods. This aligns with theoretical expectations: IMDB inputs are substantially longer, and Transformer attention scales quadratically with sequence length.

Summary. LoRA delivers the strongest accuracy and competitive latency, making it the most robust method in low-budget tuning settings. Sparse LoRA trades expressive power for parameter reduction, performing well on short inputs but degrading on IMDB. HiRA improves over Sparse LoRA by enriching adaptation rank but incurs modest inference-time overhead. Together, the results highlight the practical trade-offs among sparsity, rank, and computational structure in efficient fine-tuning.

3.3 DISCUSSION

The experiments reveal a clear trade-off between accuracy and efficiency across the evaluated PEFT methods. LoRA achieves the strongest overall performance on both SST-2 and IMDB, reflecting the stability and expressiveness of low-rank updates. Sparse LoRA offers substantial parameter reduction but incurs notable accuracy degradation, especially on IMDB, indicating that aggressive sparsification can restrict the capacity required for long-form sentiment reasoning. HiRA performs moderately well but still trails standard LoRA due to the additional complexity introduced by Hadamard modulation.

Although full latency measurements are reported separately, the expected inference-time behavior follows directly from model structure: **LoRA** $\approx$ **Sparse LoRA** $<$ **HiRA**. Sparse LoRA does not yield significant runtime gains since common PEFT implementations do not exploit sparsity during inference, while HiRA introduces extra elementwise operations that increase computational cost.

The SVD-guided analysis further shows that LoRA adapters are intrinsically low-dimensional. Compressing each adapter to retain 90% spectral energy reduces the effective rank by approximately 70% while maintaining—and for IMDB, slightly improving—task performance. This demonstrates that a fixed global rank is suboptimal and that layer-wise, data-driven rank allocation is a more principled strategy.

Overall, LoRA remains the most reliable PEFT method under constrained resources, while Sparse LoRA and HiRA illustrate the limitations of extreme sparsity and modulation-heavy designs. The spectral findings highlight promising directions for adaptive capacity control and post-hoc compression in future PEFT frameworks.

4 PROPOSED METHOD: SVD-GUIDED ADAPTIVE RANK COMPRESSION

LoRA achieves good parameter efficiency, but its performance depends strongly on the global rank hyperparameter r . Typically, a single rank (e.g., $r = 8$) is used for all modules and layers, even though the intrinsic dimensionality of task-specific updates can vary widely across layers. As a result, some layers are over-parameterized and others under-parameterized, motivating an adaptive mechanism that automatically lowers the rank when many learned directions are redundant.

We propose a simple two-stage procedure that leverages the spectral structure of LoRA updates: (i) train a standard LoRA model with a sufficiently large rank, (ii) perform singular value decomposition (SVD) on each learned LoRA weight matrix, and (iii) prune small singular directions while preserving at least 90% of the spectral energy in each module. The compressed adapters are then fine-tuned for several epochs. In our DistilBERT experiments, this reduces the average LoRA rank from 8 to about 2.4 across 12 attention modules, keeping roughly 30% of the original adapter parameters (a $\sim 70\%$ reduction).

4.1 BACKGROUND: SINGULAR VALUE DECOMPOSITION AND LOW-RANK APPROXIMATION

For any matrix $W \in \mathbb{R}^{m \times n}$, the singular value decomposition (SVD) expresses

$$W = U\Sigma V^\top,$$

where the diagonal entries of Σ , $\sigma_1 \geq \dots \geq \sigma_r > 0$, represent the singular values of W . The rank- k truncated SVD,

$$W_k = U_k \Sigma_k V_k^\top,$$

keeps only the top k singular directions and, by the Eckart–Young–Mirsky theorem, is the optimal rank- k approximation in both spectral and Frobenius norms:

$$\|W - W_k\|_F^2 = \sum_{i=k+1}^r \sigma_i^2.$$

The explained variance ratio,

$$\eta_k = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2},$$

measures how much of the matrix’s total energy is captured by the top- k components. When singular values decay rapidly, η_k becomes large for small k , indicating that the matrix is effectively low-rank—an observation that motivates low-rank adaptation methods such as LoRA.

4.2 LORA UPDATES AND THEIR SPECTRAL STRUCTURE

Recall that LoRA parameterizes the update to a pre-trained weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ as

$$\Delta W = \frac{\alpha}{r} BA, \quad (1)$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are trainable low-rank factors, r is the adapter rank, and α is a scaling coefficient. By construction, ΔW has rank at most r . After training, the learned LoRA update for a given module (e.g., an attention projection) is thus a specific matrix ΔW^* of rank at most r .

Even though ΔW^* is nominally rank r , its singular values $(\sigma_1, \dots, \sigma_r)$ need not all be equally important. In practice we observe a strong concentration of spectral energy in the leading singular values: a small number of dominant directions account for most of the Frobenius norm $\|\Delta W^*\|_F^2 = \sum_{i=1}^r \sigma_i^2$, while the remaining directions correspond to fine-grained corrections or noise. This suggests that one can further compress a trained LoRA adapter by discarding directions associated with small singular values and reparameterizing the adapter using a smaller effective rank.

4.3 SVD-GUIDED ADAPTIVE RANK SELECTION

Building on the above intuition, we propose an SVD-guided adaptive rank selection scheme that operates on a trained LoRA model. Let $\mathcal{L}$ denote the set of LoRA modules (e.g., attention projections) in the network. Our method proceeds in three stages.

Stage I: High-rank warm-up training. We first train a standard LoRA model with a fixed, moderately large rank r_{max} (e.g., $r_{\text{max}} = 8$) for T_1 epochs. During this stage, the base model parameters θ_{base} remain frozen and only the LoRA factors (A_ℓ, B_ℓ) and task-specific heads are updated, just as in conventional LoRA fine-tuning. This yields a set of learned LoRA updates

$$\Delta W_\ell^* = \frac{\alpha}{r_{\text{max}}} B_\ell A_\ell \quad \text{for each } \ell \in \mathcal{L}. \quad (2)$$

Stage II: Per-module SVD and rank pruning. For each LoRA module $\ell \in \mathcal{L}$, we compute the SVD of its learned update matrix:

$$\Delta W_\ell^* = U_\ell \Sigma_\ell V_\ell^\top, \quad (3)$$

where $\Sigma_\ell = \text{diag}(\sigma_{\ell,1}, \dots, \sigma_{\ell,r_\ell})$ and $r_\ell \leq r_{\text{max}}$. We then compute the cumulative explained variance ratio $\eta_{\ell,k}$:

$$\eta_{\ell,k} = \frac{\sum_{i=1}^k \sigma_{\ell,i}^2}{\sum_{i=1}^{r_\ell} \sigma_{\ell,i}^2}. \quad (4)$$

Given a target energy threshold τ (in our experiments, $\tau = 0.9$), we select the smallest rank k_ℓ such that

$$\eta_{\ell,k_\ell} \geq \tau. \quad (5)$$

Intuitively, k_ℓ is the minimal number of singular directions required to retain at least $\tau \times 100\%$ of the Frobenius-norm energy of ΔW_ℓ^* . The threshold τ is a single hyperparameter controlling the global trade-off between compression and approximation error.

We then construct a compressed update

$$\widetilde{\Delta W}_\ell = U_{\ell,k_\ell} \Sigma_{\ell,k_\ell} V_{\ell,k_\ell}^\top, \quad (6)$$

where U_{ℓ,k_ℓ} and V_{ℓ,k_ℓ} contain the first k_ℓ columns of U_ℓ and V_ℓ , and Σ_{ℓ,k_ℓ} is the leading $k_\ell \times k_\ell$ block of Σ_ℓ .

To express $\widetilde{\Delta W}_\ell$ again in LoRA form with a reduced rank, we define new factors

$$B'_\ell = U_{\ell,k_\ell} \Sigma_{\ell,k_\ell}, \quad A'_\ell = V_{\ell,k_\ell}^\top, \quad (7)$$

Table 2: Test performance of standard LoRA and our SVD-guided adaptive-rank LoRA on SST-2 and IMDB. All models are fine-tuned from the same DistilBERT backbone with identical optimization hyperparameters.

Method	SST-2		IMDB	
	ACC	F1	ACC	F1
LoRA	0.8922	0.8967	0.8681	0.8715
Ours	0.8933	0.8961	0.8812	0.8924

so that

$$\widetilde{\Delta W}_\ell = B'_\ell A'_\ell, \quad B'_\ell \in \mathbb{R}^{d_{\text{out}} \times k_\ell}, \quad A'_\ell \in \mathbb{R}^{k_\ell \times d_{\text{in}}}. \quad (8)$$

We then replace the original LoRA factors (A_ℓ, B_ℓ) in the model by (A'_ℓ, B'_ℓ) , and update the corresponding rank hyperparameter from r_{max} to k_ℓ . In practice, we also absorb the scaling factor α into B'_ℓ so that the implementation continues to use the same (α, r) interface as the original LoRA code.

This procedure assigns a potentially different rank k_ℓ to each LoRA module ℓ , automatically adapting the capacity to the spectral complexity of its learned update. As shown in the SVD analysis, most attention projections in DistilBERT can be reduced from rank 8 to effective ranks in $\{2, 3, 4\}$ while still retaining at least 90% of the variance, resulting in an average adaptive rank of 2.42 and an overall parameter ratio of about 30% compared with the original LoRA configuration.

Stage III: Post-compression fine-tuning. After replacing all LoRA modules with their compressed versions, we resume fine-tuning for an additional T_2 epochs. This second phase allows the model to adjust within the reduced-rank parameter space and to compensate for any small approximation error introduced by truncation. During this stage, the base model parameters remain frozen, and only the compressed LoRA factors (A'_ℓ, B'_ℓ) and task heads are updated. Empirically, a small number of additional epochs suffices to recover and often even improve performance relative to the original high-rank LoRA model, while operating with substantially fewer trainable parameters.

4.4 EXPERIMENTAL RESULTS

In this section we empirically evaluate the proposed SVD-guided adaptive rank compression for LoRA (denoted as Ours) and compare it to the standard LoRA baseline under the shared experimental protocol described in Section 3.1. We report downstream task performance on SST-2 and IMDB, as well as detailed statistics on the per-module ranks and retained variance after SVD-based compression.

Downstream Task Performance Table 2 summarizes the test performance of the baseline LoRA model and our method on the SST-2 and IMDB sentiment classification benchmarks. For each dataset, we report accuracy (ACC) and macro-averaged F1 score.

On SST-2, our method achieves 0.8933 accuracy, slightly above the LoRA baseline (0.8922), with essentially identical F1 (difference $< 10^{-3}$). Thus, SVD-based compression preserves performance on this relatively easy sentence-level task while cutting adapter parameters by nearly 70%.

On the harder IMDB dataset, improvements are larger: accuracy rises from 0.8681 to 0.8812 and F1 from 0.8715 to 0.8924 (about +2.1 F1 points). This suggests SVD-guided rank selection does more than prune redundancy: by removing low-energy directions and fine-tuning in the reduced subspace, the model learns a better-conditioned, more task-aligned representation, improving generalization on long, noisy reviews.

Overall, the method can match or surpass standard LoRA while using far fewer trainable parameters. The IMDB results in particular show that aggressive rank reduction need not reduce expressivity when discarded directions are mainly low-variance components of the LoRA updates.

Rank Compression and Retained Variance To analyze these efficiency gains, we examine the singular value spectra of each LoRA update and the adaptive ranks chosen by our method. Table 3 reports per-module ranks for all 12 LoRA modules on the attention query and value projections

Table 3: Per-module adaptive rank and retained variance for the DistilBERT attention LoRA modules after SVD-guided compression. All modules start from rank 8. The average adaptive rank is 2.42, corresponding to using only $\approx 30.2\%$ of the original LoRA parameters.

Module	Original rank	Adaptive rank	Retained variance (%)
attention.q_lin (1)	8	2	92.01
attention.v_lin (2)	8	2	94.00
attention.q_lin (3)	8	3	94.94
attention.v_lin (4)	8	2	90.33
attention.q_lin (5)	8	2	91.44
attention.v_lin (6)	8	3	93.16
attention.q_lin (7)	8	2	98.14
attention.v_lin (8)	8	3	97.82
attention.q_lin (9)	8	2	99.03
attention.v_lin (10)	8	2	96.81
attention.q_lin (11)	8	2	99.98
attention.v_lin (12)	8	4	93.28

in DistilBERT, plus the percentage of variance retained after truncation. All modules start from a uniform rank of 8 in the initial LoRA model.

We observe that for all modules the truncated SVD retains at least 90% of the Frobenius-norm energy of the original LoRA update, with several modules preserving more than 98%. At the same time, the effective ranks are dramatically reduced: most query and value projections can be represented with rank 2 or 3, and only a single module requires rank 4. The resulting average adaptive rank is 2.42, which corresponds to a parameter ratio of

$$\rho = \frac{\sum_{\ell} k_{\ell}}{8 \times 12} \approx 0.302,$$

i.e., our compressed adapters use only about 30.2% of the parameters of the original LoRA configuration.

The combination of Tables 2 and 3 highlights the central advantage of the proposed approach: by leveraging the spectral structure of the learned LoRA updates, we can discard a large number of low-energy directions while maintaining almost all of the functional capacity required for the downstream tasks. The subsequent fine-tuning stage in the reduced-rank subspace further adapts these compact adapters, leading to equal or better predictive performance compared with the full-rank LoRA baseline. This demonstrates that SVD-guided adaptive rank selection is an effective mechanism for pushing the parameter-efficiency frontier of LoRA-style methods without sacrificing accuracy.

4.5 RATIONALE AND DISCUSSION

Our SVD-guided LoRA compression leverages the fact that truncated SVD gives the optimal rank- k approximation in Frobenius norm. Selecting k_{ℓ} via an energy threshold ensures that the discarded components account for at most a $(1 - \tau)$ fraction of the spectral energy:

$$\|\Delta W_{\ell}^* - \widetilde{\Delta W}_{\ell}\|_F^2 = \sum_{i=k_{\ell}+1}^{r_{\ell}} \sigma_{\ell,i}^2 \leq (1 - \tau) \sum_{i=1}^{r_{\ell}} \sigma_{\ell,i}^2.$$

Because ranks are inferred from the singular spectra of the trained LoRA updates, the method adapts capacity per module without modifying training or adding regularizers. It is easy to integrate: train a standard LoRA model, run one SVD pass, and resume fine-tuning with the reduced ranks. The singular values also offer a simple diagnostic of layer importance; in DistilBERT, attention modules compress from rank 8 to an average of 2–3 while retaining over 90% energy. Since LoRA parameters scale linearly with rank, this yields substantial savings—about 30% of the original adapter parameters in our experiments—while preserving performance.

REFERENCES

- Jie Chen, Rui Wang, and Tao Yu. A comprehensive review of lora and its variants in parameter-efficient fine-tuning. arXiv preprint arXiv:2409.11223, 2024. URL <https://arxiv.org/abs/2409.11223>.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. Advances in neural information processing systems, 36:10088–10115, 2023.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. ICLR, 1(2):3, 2022.
- Qiushi Huang, Tom Ko, Zhan Zhuang, Lilian Tang, and Yu Zhang. Hira: Parameter-efficient hadamard high-rank adaptation for large language models. In The Thirteenth International Conference on Learning Representations, 2025.
- Samir Khaki, Xiuyu Li, Junxian Guo, Ligeng Zhu, Chenfeng Xu, Konstantinos N Plataniotis, Amir Yazdanbakhsh, Kurt Keutzer, Song Han, and Zhijian Liu. Sparselora: Accelerating llm fine-tuning with contextual sparsity. arXiv preprint arXiv:2506.16500, 2025.
- Yihan Wang, Xin Zhang, Tianle Liu, and Qiang Yang. Parameter-efficient fine-tuning for large language models: A comprehensive survey. arXiv preprint arXiv:2503.02456, 2025. URL <https://arxiv.org/abs/2503.02456>.
- Hao Zhu, Yu Zhang, and Zhiyuan Liu. A survey on low-rank adaptation methods for large language models. arXiv preprint arXiv:2501.04566, 2025. URL <https://arxiv.org/abs/2501.04566>.

Table 4: Common training hyperparameters used for all datasets and all adaptation methods.

Hyperparameter	Value
Base model	distilbert-base-uncased
Number of epochs	3
Batch size	16
Learning rate	3×10^{-4}
Max sequence length	128
Save steps	500
Evaluation steps	500
Random seed	42
Output directory	./results

A EXPERIMENTAL SETUP

Tasks, Datasets, and Base Model We consider three widely used benchmark datasets that cover both sequence classification and language modeling:

- **SST-2**: binary sentiment classification on single sentences.
- **IMDB Reviews**: binary sentiment classification on movie reviews.
- **WikiText-2**: word-level language modeling on Wikipedia articles.

For each dataset we use the standard train/test splits provided by the corresponding benchmark. All text is tokenized with the `distilbert-base-uncased` tokenizer, using the same preprocessing pipeline for all three adaptation methods.

As the common backbone, we adopt the `distilbert-base-uncased` model. For SST-2 and IMDB we attach a task-specific classification head on top of the [CLS] representation, while for WikiText-2 we use a language modeling head. In all experiments, the pre-trained DistilBERT parameters are frozen and only the task-specific adapters (LoRA, Sparse LoRA, or HiRA) and the small output heads are updated.

Training Configuration Table 4 summarizes the training hyperparameters that are shared across all datasets and methods. These correspond directly to the command-line arguments used to launch training:

```
python train_distilbert_lora.py \
  --dataset $DATASET \
  --model_name distilbert-base-uncased \
  --output_dir ./results \
  --epochs 3 \
  --batch_size 16 \
  --learning_rate 3e-4 \
  --lora_r 8 \
  --lora_alpha 16 \
  --lora_dropout 0.1 \
  --max_length 128 \
  --save_steps 500 \
  --eval_steps 500 \
  --seed 42
```

We fine-tune for 3 epochs with a batch size of 16 and a learning rate of 3×10^{-4} . The maximum input sequence length is truncated or padded to 128 tokens. We set the random seed to 42 for all runs in order to control for stochastic variation in initialization, data shuffling, and optimization.

All three methods use adapters with the same intrinsic rank and scaling. Specifically, we set the adapter rank to $r = 8$, the scaling factor to $\alpha = 16$, and apply a dropout rate of 0.1 on the adapter pathway.

Table 5: Adapter hyperparameters for LoRA, Sparse LoRA, and HiRA. All methods use the same nominal rank, scaling, and dropout to ensure a fair comparison.

Method	Rank r	Scaling α	Adapter dropout
LoRA	8	16	0.1
Sparse LoRA	8	16	0.1
HiRA	8	16	0.1

For LoRA this corresponds directly to the low-rank decomposition parameters. Sparse LoRA and HiRA are configured so that their effective low-rank components share the same (r, α) and dropout as LoRA, thereby matching capacity as closely as possible. The shared adapter hyperparameters are summarized in Table 5.

Fair Comparison Protocol To ensure that any observed performance difference can be attributed to the adaptation mechanism rather than confounding factors, we adhere to the following protocol:

- **Same backbone and initialization.** All methods start from the same pre-trained `distilbert-base-uncased` checkpoint, with identical tokenizer and vocabulary.
- **Same data and preprocessing.** For each dataset, we use the same training and test splits and apply identical tokenization, truncation to length 128, and padding strategy.
- **Same optimization hyperparameters.** The learning rate, batch size, number of epochs, evaluation frequency, and random seed are fixed as in Table 4 for all methods and datasets.
- **Matched adapter capacity.** The adapter rank r , scaling α , and dropout are kept identical across LoRA, Sparse LoRA, and HiRA (Table 5), so that they operate under comparable parameter budgets.
- **Task-specific heads.** For each dataset, the same task head architecture (classification head or language modeling head) is used for all methods, and only the adapter type is changed.
- **Evaluation metrics.** On SST-2 and IMDB we report classification accuracy on the test set, while on WikiText-2 we report token-level perplexity. For a given dataset, the same metric and evaluation code are used for all methods.

Under this setup, we can view each method as learning an adapter θ_{adapt} over a fixed backbone θ_{base} that minimizes the same empirical risk

$$\mathcal{L}(\theta_{\text{adapt}}) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i; \theta_{\text{base}}, \theta_{\text{adapt}}), y_i), \quad (9)$$

where ℓ denotes cross-entropy for both classification and language modeling. Because θ_{base} and the optimization hyperparameters are shared, the performance differences observed in Section 4.4 can be attributed primarily to how each method parameterizes and trains θ_{adapt} (low-rank, sparse low-rank, or Hadamard high-rank), rather than to extraneous experimental choices.

Implementation of Sparse LoRA In this project we implement a simplified version of Sparse LoRA that preserves the core idea of enforcing structured sparsity on LoRA’s low-rank updates, but does not reproduce the full algorithmic stack proposed in the official Sparse LoRA release by Z-Lab.¹ Our goal is to study the empirical trade-offs between dense low-rank adaptation and sparse low-rank adaptation under a unified experimental pipeline, rather than to replicate the full production-level implementation of Sparse LoRA.

Sparsity Masking. Given a LoRA update of the form

$$\Delta W = BA,$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$, our simplified Sparse LoRA applies elementwise masking to induce sparsity:

$$\Delta W_{\text{sparse}} = M \odot (BA),$$

¹<https://github.com/z-lab/sparselora>

where the binary mask M is constructed by selecting the top- $k\%$ magnitude elements of ΔW during initialization and keeping this mask fixed throughout training. Unlike the official implementation, we do not perform iterative sparsification, scheduled pruning, nor structured block-sparse training. This design allows Sparse LoRA to remain lightweight and easy to integrate with the standard HuggingFace LoRA pipeline.

Training Protocol Compatibility. Our Sparse LoRA shares the same:

- adapter rank $r = 8$ and scaling factor $\alpha = 16$,
- dropout rate of 0.1 on the adapter pathway,
- optimizer, batch size, and 3-epoch training schedule,

as used for LoRA and HiRA (Tables 4–5). This ensures that observed performance differences stem primarily from the introduction of sparsity, rather than from changes in capacity or training conditions.

Intended Behavior and Limitations. Our simplified Sparse LoRA enforces sparsity on the effective update matrix ΔW but does not implement:

- activation sparsity or structured group sparsity,
- dynamic mask updates during training,
- any CUDA kernels enabling sparse matrix acceleration.

Consequently, the method reduces the number of trained parameters, but does not provide actual inference-time speedup on modern hardware and libraries, which explains the latency results observed in Section 4.4. Instead, our experiments isolate the effect of parameter sparsity on task accuracy, representational capacity, and stability during fine-tuning.

Summary. The simplified Sparse LoRA implementation used in this work preserves the conceptual objective of learning sparse low-rank adaptation while remaining compatible with the standard LoRA codebase. It enables a clean, controlled comparison against LoRA and HiRA, while illustrating the limitations of naive sparsification in the absence of specialized sparse inference support.